

Campus Service Centre — Part C:

Algorithm Experiment

Experimental comparison of Selection Sort, Insertion Sort, Merge Sort and Quick Sort using randomly generated integer arrays of size 20, 50, 100 and 500, plus an additional almost-sorted 100-element test. For each run every algorithm receives an identical copy of the same array. Only comparisons between data values are counted, and only the sorting call itself is timed using `System.nanoTime()`.

1. Results — Random Arrays

Number of comparisons (deterministic for the seeded generator):

Algorithm	n = 20	n = 50	n = 100	n = 500
Selection Sort	190	1225	4950	124750
Insertion Sort	107	586	2693	64246
Merge Sort	61	223	541	3844
Quick Sort	69	237	645	4659

Execution times are machine-dependent; the Java program prints the measured nanosecond timings directly in its output table.

2. Additional Test — Almost-Sorted 100-element Array

The 100-element array was first sorted ascending, then five neighbouring pairs (0-1, 2-3, 4-5, 6-7, 8-9) were swapped.

Algorithm	Comparisons
Selection Sort	4950
Insertion Sort	103
Merge Sort	357
Quick Sort	4930

3. Answers to the Questions

Which algorithm performed the fewest comparisons?

Merge Sort. It recorded the lowest comparison count at every input size (61, 223, 541, 3844), consistently close to $n \log n$. Quick Sort was a close second.

Which algorithm performed the most comparisons?

Selection Sort. It always performs $n(n-1)/2$ comparisons regardless of the data (190, 1225, 4950, 124750), which is $O(n^2)$.

Which algorithm performed best on the almost-sorted array?

Insertion Sort. It needed only 103 comparisons (about n), because it is adaptive: it detects that the array is already nearly ordered. Selection Sort and Quick Sort both still needed about 4950 comparisons.

Which algorithm appeared to scale best as the input became larger?

Merge Sort. It has guaranteed $O(n \log n)$ behaviour and stayed the fastest-growing but lowest comparison count at $n = 500$. Quick Sort also scaled well on random data but with slightly more comparisons.

Did the experimental results generally agree with the theoretical time complexity? Explain.

Yes. Selection Sort matched $n(n-1)/2$ exactly ($O(n^2)$). Insertion Sort averaged about $n^2/4$ comparisons on random data ($O(n^2)$) and about n on the almost-sorted array ($O(n)$ best case). Merge Sort grew as $n \log n$, and Quick Sort grew as $n \log n$ on random data but degraded to $O(n^2)$ on the almost-sorted array — all exactly as theory predicts.

Is execution time alone enough to decide which algorithm is better? Explain.

No. Execution time depends on the hardware, the language, and JVM/JIT warm-up, so it is not portable. It also hides constant factors and data movement: for example Merge Sort does $O(n)$ extra copying to a temporary array that costs time but no comparisons, while Insertion Sort performs many shifts. Comparison count is portable but still ignores swaps, extra memory ($O(n)$ for Merge Sort, in-place for the others), stability, and best/worst-case behaviour. A good decision should consider comparisons, time, space and stability together, for the type of data actually expected.